Capstone Scope Document

Capstone Scope Document

Problem Statement

kk-payments currently deploys to a single environment: the `kijani-project` namespace on a local Minikube cluster (Week 9). There is no staging environment, so engineers testing changes against a production-shaped configuration have no isolation, and a bad deploy is only caught after it reaches production. Separately, the Week 10 serverless receipt chain (`kk-receipts` → `kk-processor` → `kk-notifier`) has never been wired to the actual kk-payments service — nothing in the Kubernetes layer writes the S3 object that fires it, so the "receipt on every payment" flow has never been demonstrated end-to-end. The capstone closes both gaps: it adds an isolated staging namespace with its own configuration and a Jenkins gate in front of production, and it connects kk-payments' `/pay` endpoint to the receipt chain so a payment in staging produces a notified receipt.

Track

Track A (Infrastructure-First)

What I Will Build

- **Terraform** module that provisions the `kijani-staging` namespace, isolated from `kijani-project`, with the receipts bucket name defined once as the shared source of truth.
- **Ansible** playbook that renders and applies the staging-specific `kk-payments` ConfigMap (different `DB_HOST`, `LOG_LEVEL`, `RECEIPTS_BUCKET`) into `kijani-staging`, using the same Deployment manifest as production.
- **Jenkins pipeline** that on merge to `main` provisions staging infra, deploys `kk-payments` to staging, runs an automated smoke test (health check + a real `/pay` call that must produce a receipt), and only then offers a production approval gate that records who approved and why.
- **Serverless receipt chain** (`kk-receipts` → `kk-processor` → `kk-notifier`) wired so `kk-payments`' `/pay` endpoint in staging writes to `kijani-payments-receipts-staging` and the chain fires end-to-end, ending in a structured log line.
- **Log-based error rate monitoring** (Option B) reading kk-payments' structured JSON logs and alerting when the 5xx rate exceeds 5% over a 2-minute window, gating the Jenkins approval stage.

What Is Out of Scope

- **Real cloud deployment of the serverless chain** — the chain runs against a local S3 mock (`/tmp/kijani-s3-local`) and `serverless-offline`, not a real AWS account, because Track A's monitoring/staging requirements are the depth target for this submission, not cloud serverless deployment (that is Track B's focus).
- **Prometheus (Option A)** — no Prometheus instance is installed in this environment; Option B (log-based) is used instead, per the equivalence project.md draws between the two options.
- **A managed staging database** — `DB_HOST` in the staging ConfigMap points at a Kubernetes Service name (`kk-postgres.kijani-staging...`) that is not itself provisioned by this capstone; kk-payments does not currently make a real DB connection (Week 9 carry-forward), so provisioning a staging Postgres instance would add infrastructure without a consumer.
- **kk-api staging deployment** — only `kk-payments` is required to run in staging per the track brief; `kk-api` remains production-only (Week 9 manifests, unchanged).

Success Criteria

1. `terraform apply` from a clean checkout creates the `kijani-staging` namespace, verifiable with `kubectl get ns kijani-staging`.
2. A merge to `main` triggers the Jenkins pipeline without manual steps up to the approval gate; the gate blocks production deploy until the staging smoke test (`scripts/smoke-test.sh`) has exited 0.
3. A `POST /pay` request against `kk-payments` in staging produces a matching receipt file under `kijani-payments-receipts-staging`, and within a few seconds a `kk-notifier` log line reports the same `orderId` — demonstrable live by tailing `node scripts/run-local-chain.js staging` during the demo.

Architecture Diagram

See `docs/architecture.svg` / `docs/architecture.png` (embedded in `README.md`).